

Guía de trabajo y facilitación: literatura con agentes

Para aprender, practicar y enseñar búsqueda reproducible, compuertas humanas y evidencia auditable

La frase que organiza todo: el agente propone, el código comprueba lo comprobable y una persona firma lo que cuenta como evidencia.

Esta guía está diseñada para dos usos: el estudiante puede leerla y ejecutar el recorrido; quien enseña puede leerla una vez y después usar la última página como tarjeta de cabina. El laboratorio no intenta mostrar que un modelo "hace una revisión" por sí solo. Muestra algo más útil: cómo una búsqueda deja candidatos reproducibles y cómo un agente avanza trabajo mecánico sin recibir autoridad para decidir qué evidencia entra.

Duración: sección completa de 75 minutos; 8 de búsqueda automatizada, 19 de demo central y 2 de transferencia.

Carpeta: course/exercises/literature_agent_lab/

Requisitos: Python 3.9 o superior. El núcleo funciona sin red; la búsqueda tiene una fixture offline. Codex Desktop, Codex en VS Code o Claude Code son opcionales.

Aviso que debe decirse en voz alta: los ocho registros/DOI del tamizaje son reales. Las cuatro fichas T1-T4 y sus pasajes son sintéticos y didácticos; no son citas de artículos reales. Nunca mezcle ambos conjuntos como si fueran una misma fuente de evidencia.

1. Cómo usar esta guía

Si eres estudiante

Lee las páginas 1-4 antes de la sesión. Al terminar debes poder explicar la diferencia entre descubrir, cribar, extraer y verificar. Si quieres practicar, ejecuta primero la búsqueda offline: produce todos los artefactos sin depender de la red y sin modificar el caso congelado.

Durante la clase, no evalúes al agente por la fluidez de su texto. Observa qué archivos lee, qué archivos puede escribir, cuándo se detiene y qué evidencia acompaña cada afirmación.

Si enseñas la sesión

Lee la guía completa una vez. Ensaya las páginas 5-10 en una copia del laboratorio. Durante la clase puedes enseñar desde la página 13. Los bloques **Decir** son formulaciones sugeridas, no texto obligatorio; los comandos y resultados esperados sí deben mantenerse exactos.

Lo que construirán

```
pregunta y protocolo
-> estrategia de búsqueda aprobada
-> respuesta original de OpenAlex
-> candidatos normalizados y deduplicados
-> propuestas de cribado y evidencia
-> decisiones humanas
-> ledger oficial
-> brief ensamblado
```

Criterio de éxito: al finalizar, otra persona puede reconstruir de dónde salió una afirmación sin leer el chat que produjo el trabajo.

2. El mapa mental que necesitas

Un chat suele mezclar instrucciones, borradores y decisiones en una sola conversación. Este laboratorio los separa en artefactos visibles:

- `case/`: pregunta, protocolo y paquete de fuentes congelados.
- `work/`: propuestas de la IA. Son editables y todavía no cuentan como evidencia.
- `human/`: decisiones firmadas por una persona.
- `official/`: ledger oficial; solo recibe filas a través de la compuerta humana.
- `output/`: brief generado exactamente desde el ledger.
- `STATE.json` y `TRACE.csv`: estado y rastro que permiten reanudar sin depender del chat anterior.

Decir: "No estamos automatizando el juicio. Estamos haciendo explícito dónde ocurre y dejando que la automatización prepare el trabajo que llega a ese punto."

Agente, skill, loop y compuerta

- **Agente:** un modelo que puede observar archivos, elegir acciones, usar herramientas y dejar resultados persistentes. No es solo una respuesta: actúa dentro de un entorno.
- **Skill:** una receta reutilizable para una tarea delimitada, por ejemplo extraer un pasaje con localizador. El agente puede usar varias skills.
- **Loop:** observar estado, actuar, comprobar el resultado y decidir el siguiente paso. El loop produce continuidad; no produce autoridad.
- **Validador:** código determinista que detecta reglas mecánicas: localizadores exactos, campos faltantes o verbos causales prohibidos.
- **Compuerta humana:** el único paso que convierte una propuesta en una decisión oficial.

La diferencia clave es entre **detectar forma** y **juzgar significado**. El código puede ver que un localizador no coincide. No puede decidir con seguridad que "intención de compartir" equivale a "conducta de compartir". Ahí sigue siendo indispensable una lectura humana.

3. Búsqueda automatizada: qué hace y qué no

OpenAlex es una tool de descubrimiento bibliográfico. Recibe una estrategia y devuelve metadatos de trabajos. No sabe cuáles responden nuestra pregunta, no verifica el texto completo y no decide qué entra al brief.

El mini-laboratorio separa cinco artefactos:

- `search_strategy.json`: términos, filtros, límites, huecos conocidos y regla de parada, escritos antes de buscar.
- `output/raw/`: respuesta original que permite auditar qué devolvió la fuente.
- `output/candidates.csv`: metadatos planos y legibles para el cribado posterior.
- `output/dedup_report.csv`: duplicados detectados por DOI, OpenAlex ID o título normalizado.
- `output/search_log.json`: consulta, fecha, modo, fuente usada, topes y conteos.

Distinción obligatoria: una fila en `candidates.csv` significa "la búsqueda la recuperó". No significa "cumple el protocolo", "fue leída" ni "es evidencia".

Por qué conservar la respuesta original

Los índices cambian y el orden de resultados puede variar. Guardar la respuesta permite distinguir entre un cambio en la base y un cambio en nuestro código. El modo offline usa una respuesta registrada y deja esa procedencia explícita en el log.

Un límite honesto

La práctica usa una base, términos principalmente en inglés, artículos recientes y un tope pequeño. Sirve para aprender el recorrido; no alcanza para declarar una revisión exhaustiva o sistemática.

4. La misma práctica en Codex Desktop y VS Code

Ruta A: Codex Desktop

1. Abre la carpeta `course/exercises/literature_agent_lab/` como proyecto y entra en `discovery/` desde la terminal. Así el agente puede leer también `case/question.md` sin salir del espacio de trabajo. 2. Pide al agente: "Lee `discovery/README.md`, `discovery/search_strategy.json` y `case/question.md`. No ejecutes nada. Resume la pregunta, los tres grupos de consulta, los límites y los huecos conocidos. Espera confirmación." 3. Revisa la estrategia. La consulta se aprueba antes de ejecutar la tool. 4. Abre la terminal del proyecto y ejecuta el comando `offline`. 5. Revisa los cuatro artefactos y el diff; no copies resultados directamente al caso congelado.

Ruta B: VS Code

1. Abre exactamente la misma carpeta: `course/exercises/literature_agent_lab/`. 2. Usa Codex o Claude Code con el mismo encargo de lectura. 3. Ejecuta los mismos comandos en la terminal integrada. 4. Abre `discovery/output/candidates.csv`, `discovery/output/dedup_report.csv` y `discovery/output/search_log.json` en pestañas separadas.

```
cd discovery

# Reproducible y sin red
python3 scripts/search_openalex.py --offline

# Intenta OpenAlex y cae visiblemente a la fixture si falla
python3 scripts/search_openalex.py
```

Pregunta de reflexión: ¿qué decisión metodológica queda escondida si solo mostramos `candidates.csv` y borramos `search_strategy.json` y `search_log.json`?

La respuesta esperada: quedarían ocultos los términos, filtros, cobertura, fecha, topes y fuente de los resultados; no podríamos interpretar ausencias ni reproducir el descubrimiento.

5. Preparación: una vez el día anterior

Desde la raíz del repositorio:

```
cd course/exercises/literature_agent_lab
python3 scripts/test_lab.py
cd discovery
python3 scripts/test_discovery.py
```

Debes ver **33 comprobaciones del laboratorio principal** y **25 pruebas offline de descubrimiento**. Luego ensaya el recorrido completo en una copia del laboratorio, no en la carpeta que usarás durante la clase.

Checklist de cinco minutos

- Agranda la terminal hasta que se lean 80-100 caracteres por línea.
- Abre `discovery/search_strategy.json`, `PROMPTS.md`, `STATE.json` y `TRACE.csv` en pestañas distintas. `output/review_brief.md` se abrirá solo después de ensamblarlo en el minuto 68.
- Decide el nombre que escribirás en `--by`, por ejemplo `Kristian`.
- Copia los comandos de esta guía a un bloc de notas sin formato.
- Confirma que la terminal permite interacción: `review_gate.py` exige retectar el ID.
- Cierra notificaciones y cualquier archivo con datos privados.

Preparar el punto de partida

```
cp -R fixtures/waiting_gate/. .
python3 scripts/validate_review.py
```

Este fixture representa el momento posterior al trabajo inicial del agente: hay propuestas en `work/`, todavía no hay decisiones de evidencia en `human/`, y el validador bloquea. Es el mejor punto de partida porque concentra el demo en lo pedagógicamente importante.

No ocultar: di que usas un estado preparado para que todos vean la misma situación. El agente sí puede participar en vivo al leer, corregir y reanudar; no necesitas esperar a que procese doce registros durante la clase.

6. Minutos 54-58: contrato y primer diagnóstico

Objetivo

Que la audiencia vea que el agente tiene capacidad operativa, pero no permiso para decidir.

Mostrar

Abre `AGENTS.md` y señala tres líneas: puede escribir en `work/`, no puede editar `human/` ni `official/`, y no puede ejecutar `review_gate.py`.

Si usarás Codex o Claude Code, inícialo dentro de la carpeta del laboratorio y pega:

```
Lee AGENTS.md, STATE.json, las últimas filas de TRACE.csv y work/.
No edites nada. Dime qué trabajo ya está propuesto, qué decisiones humanas
faltan y cuál es la próxima acción permitida. Espera confirmación.
```

Decir mientras responde

"Miren lo que no le pedí: no le pedí decidir. Un buen encargo define artefactos, autoridad y condición de parada, no solo una tarea."

Abre `work/evidence_pending.csv` y luego `human/evidence_verifications.csv`.

"El trabajo puede estar avanzado sin estar aprobado. `work/` contiene propuestas; `human/` sigue vacío. Esa separación evita que una frase plausible se convierta silenciosamente en evidencia."

Si el agente tarda

No esperes más de 30 segundos. Interrúmpelo, muestra los dos CSV y continúa. El aprendizaje está en los artefactos, no en ver aparecer tokens.

Señal de avance: la audiencia puede responder: "¿Quién puede escribir en el ledger?" Respuesta: solo la compuerta ejecutada por una persona.

7. Minutos 58-63: el bloqueo es el resultado correcto

Ejecuta:

```
python3 scripts/validate_review.py
```

La salida relevante debe ser:

```
BLOCKED
- causal-language: EV-T3 uses 'redujo' but T3 declares
  'estudio observacional'
- anchor: EV-T4 locator does not match T4
- gate: human verification pending for EV-T1, EV-T3, EV-T4
```

Decir: "BLOCKED no significa que el sistema falló. Significa que se negó a producir certeza antes de tener evidencia y firma suficientes."

Aprueba la fila limpia:

```
python3 scripts/review_gate.py --kind evidence --id EV-T1 \
  --decision verified --by "Kristian"
```

La terminal preguntará:

```
Retype EV-T1 to record 'verified':
```

Escribe **EV-T1** y pulsa Enter.

Decir durante la pausa: "Esta fricción es deliberada. Reescribir el identificador transforma un clic reflejo en un acto visible de autoría. No prueba que la decisión sea correcta, pero sí deja claro quién la tomó."

Qué acaba de ocurrir

La fila salió de `work/evidence_pending.csv`, apareció una verificación en `human/` y entró al ledger en `official/`. El agente no hizo ese traslado. El script también actualizó la huella de las decisiones y el rastro.

8. Minutos 63-68: dos errores, dos tipos de control

Rechaza las dos filas problemáticas. Cada comando exigirá retectar su ID.

```
python3 scripts/review_gate.py --kind evidence --id EV-T3 \  
--decision rejected \  
--correction "Diseño observacional: use lenguaje asociativo" \  
--by "Kristian"
```

```
python3 scripts/review_gate.py --kind evidence --id EV-T4 \  
--decision rejected \  
--correction "La fuente mide intención declarada, no conducta" \  
--by "Kristian"
```

El contraste que debes enseñar

EV-T3: error mecánicamente detectable. La ficha declara un estudio observacional, pero la interpretación dice que el asistente "redujo" algo. El validador puede cruzar diseño y una lista de verbos causales. Es una alarma lexical útil, no comprensión causal.

EV-T4: error semántico. El código también detecta que el localizador es incorrecto. Pero una persona debe notar el salto más importante: el estudio midió intención declarada de compartir, no la conducta de compartir. Un localizador corregido no arreglaría esa interpretación.

Decir: "EV-T3 muestra qué conviene mecanizar. EV-T4 muestra por qué no debemos confundir validación automática con lectura crítica."

Pedir la corrección al agente

Lee STATE.json, TRACE.csv, human/evidence_verifications.csv y las fichas T3-T4. Repropón EV-T3 y EV-T4 con los mismos evidence_id en work/evidence_pending.csv. Corrige el verbo, el localizador y el salto de intención a conducta. Ejecuta el validador. No uses review_gate.py y detente.

9. Reanudar sin memoria conversacional

Cuando el agente termine de corregir, cierra por completo la sesión. Abre una sesión nueva y pega:

```
Lee STATE.json, las últimas filas de TRACE.csv, work/ y human/.

Resume: último ítem completo, qué decisiones humanas ya existen,
qué filas siguen abiertas y cuál es la próxima acción permitida.

No edites. Espera confirmación.
```

Decir: "La continuidad no vive en la memoria del chat. Vive en artefactos que otra sesión, otra persona o incluso otro modelo puede inspeccionar."

Confirma visualmente las dos filas corregidas. Deben conservar EV-T3 y EV-T4; una corrección no crea una identidad nueva. Luego verifica ambas:

```
python3 scripts/review_gate.py --kind evidence --id EV-T3 \
--decision verified --by "Kristian"

python3 scripts/review_gate.py --kind evidence --id EV-T4 \
--decision verified --by "Kristian"
```

Retecla el identificador en cada caso y valida:

```
python3 scripts/validate_review.py
```

Debe terminar en VALID. Si el agente no produjo filas correctas, no improvises una aprobación. Usa la escalera de recuperación de la página 11.

Qué debe recordar la audiencia

- Cambiar de modelo no debe borrar el estado del proyecto.
- Reanudar comienza leyendo, no escribiendo.
- Un rechazo es información de trabajo, no un fracaso.
- La misma evidencia puede volver a proponerse, pero necesita una nueva decisión humana.

10. Minutos 68-73: ensamblar y demostrar procedencia

Ejecuta:

```
python3 scripts/assemble_brief.py
```

La salida esperada es:

```
Wrote output/review_brief.md from 3 verified ledger rows
```

Abre el brief y señala una afirmación. Cada una incluye fuente, diseño, población, resultado, pasaje, localizador y revisor. En EV-T4 debe decir **intención declarada**, no conducta.

Decir: "El brief no es el lugar donde aparece evidencia nueva. Es una vista del ledger. Si quiero cambiar una afirmación, debo cambiar la evidencia por el recorrido autorizado, no retocar la prosa final."

Demostración de inmutabilidad pedagógica

Agrega manualmente una palabra al brief, guarda y vuelve a validar:

```
python3 scripts/validate_review.py
```

Debe bloquear con:

```
BLOCKED  
- assembly: review_brief.md was not generated exactly from the ledger
```

Restaura el archivo ejecutando de nuevo:

```
python3 scripts/assemble_brief.py
```

Cierre verbal

"No hicimos una revisión completa en diecinueve minutos. Construimos una unidad de revisión en la que cada transición deja evidencia: propuesta, comprobación, decisión y ensamblaje. Esa unidad sí puede escalar."

Si alguien dice que el sistema es rígido: "Para explorar preguntas, esta rigidez sobra. Para una afirmación que vamos a enseñar, publicar o firmar, la rigidez protege la trazabilidad."

11. Escalera de recuperación: nunca dependa del espectáculo

Nivel 1: el agente tarda o habla demasiado

Interrúmpelo. Abre directamente `work/evidence_pending.csv`, explica las propuestas y ejecuta el validador. No gastes más de 30 segundos esperando.

Nivel 2: el agente no corrige bien EV-T3 o EV-T4

Carga el estado posterior al rechazo:

```
cp -R fixtures/rejected/. .
```

Pide al agente que lea el estado y proponga correcciones. Si todavía falla, explica oralmente las correcciones y pasa al nivel 3. Nunca verifiques una fila defectuosa para salvar el ritmo.

Nivel 3: no hay modelo, red o tiempo

```
cp -R fixtures/ready/. .
python3 scripts/validate_review.py
python3 scripts/assemble_brief.py
```

Esto carga tres filas ya verificadas y permite enseñar ensamblaje y procedencia en aproximadamente un minuto. Di con claridad que cargaste un estado de contingencia.

Nivel 4: la terminal interactiva no acepta el ID

No canalices el comando por un pipe: la compuerta exige una terminal real. Abre una terminal local normal, entra a la carpeta y repite el comando. Si no puedes, muestra `human/evidence_verifications.csv` del fixture `ready` y explica la interacción.

Si te quedan solo cinco minutos

Muestra `waiting_gate`, ejecuta el validador, explica EV-T3/EV-T4, carga `ready` y ensambla. Conservas el arco conceptual completo: propuesta, bloqueo, juicio humano y salida trazable.

12. Preguntas difíciles y respuestas cortas

¿Esto es realmente un agente o solo scripts?

Los scripts son las barandas deterministas. El agente observa estado, decide qué herramienta usar y modifica artefactos permitidos. La arquitectura combina ambos; no necesita fingir que el modelo es confiable por sí solo.

¿Por qué no dejar que el agente apruebe si el validador da verde?

Porque verde significa que pasaron reglas mecánicas conocidas. EV-T4 puede tener forma correcta y significado equivocado. Cumplimiento formal no equivale a validez sustantiva.

¿La huella criptográfica impide manipulación?

No. Es una alarma pedagógica contra ediciones silenciosas, no seguridad del sistema operativo. Quien tenga acceso total puede recalcularla.

¿No es demasiado rígido para investigar?

Para explorar, sí. Para convertir una lectura en evidencia que alimentará un informe, la separación reduce ambigüedad y hace posibles la auditoría y la corrección.

¿Cómo escala a cientos de fuentes?

Cambian el almacenamiento, la recuperación y el muestreo; no cambia la arquitectura de autoridad. El agente puede buscar y preparar más filas, mientras el protocolo define qué se revisa humana o mecánicamente.

¿Por qué fuentes sintéticas?

Permiten distribuir pasajes exactos, sembrar errores conocidos y comprobar el aprendizaje sin atribuir frases falsas a artículos reales.

¿Qué generalizo a mi proyecto mañana?

Define un artefacto que la IA puede proponer, una decisión que no puede tomar, dos comprobaciones mecánicas, un archivo de estado y la evidencia que debe acompañar cada afirmación.

13. Tarjeta de cabina: enseñar desde esta página

Antes de entrar

```
cd course/exercises/literature_agent_lab
python3 scripts/test_lab.py          # 33 checks
cd discovery
python3 scripts/test_discovery.py    # 25 pruebas offline
python3 scripts/search_openalex.py --offline
cd ..
cp -R fixtures/waiting_gate/. .
```

30-38 · Descubrimiento

Mostrar: discovery/search_strategy.json, discovery/output/candidates.csv, discovery/output/dedup_report.csv y discovery/output/search_log.json.

Decir: "Descubrir produce candidatos; no produce evidencia."

En vivo: cd discovery && python3 scripts/search_openalex.py

Fallback: python3 scripts/search_openalex.py --offline

54-58 · Contrato

Mostrar: AGENTS.md, work/, human/.

Decir: "El agente propone; no firma."

Prompt: "Lee estado y rastro. No edites. Di qué falta y espera."

58-63 · Bloqueo

```
python3 scripts/validate_review.py
python3 scripts/review_gate.py --kind evidence --id EV-T1 \
--decision verified --by "Kristian"
```

Decir: "BLOCKED es el resultado correcto antes de una decisión."

63-68 · Dos errores

Rechazar EV-T3: causalidad en estudio observacional.

Rechazar EV-T4: localizador incorrecto + intención no es conducta.

Pedir corrección, cerrar sesión, reanudar leyendo STATE.json y TRACE.csv, verificar T3/T4.

68-73 · Ensamblaje

```
python3 scripts/validate_review.py
python3 scripts/assemble_brief.py
```

Editar el brief, validar, mostrar bloqueo y volver a ensamblar.

Botón de contingencia

```
cp -R fixtures/ready/. .
python3 scripts/validate_review.py
python3 scripts/assemble_brief.py
```

Frase final

"La calidad no proviene de pedirle al modelo que sea cuidadoso. Proviene de diseñar un recorrido donde cada afirmación debe mostrar de dónde salió, qué regla pasó y quién la aprobó."